

✓ Instalación de librerías

```
1 !pip install requests
2 import urllib.request
3
4 !pip install tsplib95
5 import tsplib95
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5)
Requirement already satisfied: certifi<=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (202)
Collecting tsplib95
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: Click<=6.0 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (8.3.1)
Collecting Deprecated<=1.2.9 (from tsplib95)
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl.metadata (5.7 kB)
Collecting networkx<=2.1 (from tsplib95)
  Downloading networkx-2.8.8-py3-none-any.whl.metadata (5.1 kB)
Collecting tabulate<=0.8.7 (from tsplib95)
  Downloading tabulate-0.8.10-py3-none-any.whl.metadata (25 kB)
Collecting wrapt<2,>=1.10 (from Deprecated<=1.2.9->tsplib95)
  Downloading wrapt-1.17.3-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl (10.0 kB)
  Downloading networkx-2.8.8-py3-none-any.whl (2.0 MB)
  2.0/2.0 MB 67.6 MB/s eta 0:00:00
  Downloading tabulate-0.8.10-py3-none-any.whl (29 kB)
  Downloading wrapt-1.17.3-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (88 kB)
  88.0/88.0 kB 5.4 MB/s eta 0:00:00
Installing collected packages: wrapt, tabulate, networkx, Deprecated, tsplib95
  Attempting uninstall: wrapt
    Found existing installation: wrapt 2.1.1
    Uninstalling wrapt-2.1.1:
      Successfully uninstalled wrapt-2.1.1
  Attempting uninstall: tabulate
    Found existing installation: tabulate 0.9.0
    Uninstalling tabulate-0.9.0:
      Successfully uninstalled tabulate-0.9.0
  Attempting uninstall: networkx
    Found existing installation: networkx 3.6.1
    Uninstalling networkx-3.6.1:
      Successfully uninstalled networkx-3.6.1
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This b
bigframes 2.36.0 requires tabulate>=0.9, but you have tabulate 0.8.10 which is incompatible.
scikit-image 0.25.2 requires networkx>=3.0, but you have networkx 2.8.8 which is incompatible.
mapclassify 2.10.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
momepy 0.11.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
spopt 0.7.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
Successfully installed Deprecated-1.2.18 networkx-2.8.8 tabulate-0.8.10 tsplib95-0.7.1 wrapt-1.17.3
```

```
1 import random #Libreria para generar numeros y listas aleatorias
2 import copy #Permite hacer copias de objetos en python: listas, diccionarios,...
```

✓ Carga de datos del problema

```
1 #Librerías y carga del problema
2
3 #http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
4 #Documentación :
5 # https://wwwproxy.iwr.uni-heidelberg.de/groups/comopt/software/TSPLIB95/tsp95.pdf
6 # https://tsplib95.readthedocs.io/usage.html
7 # https://tsplib95.readthedocs.io/modules.html#module-tsplib95.models
8
9 #Matriz de distancias
10 #file = "swiss42.tsp" ;
11 #urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/swiss42.tsp.gz", file
12 #!gzip -d swiss42.tsp.gz #Descomprimir el fichero de datos
13
14 # Estaba caído el server, por lo que lo hacemos usando esta URL:
15 file = "swiss42.tsp" ;
16 urllib.request.urlretrieve("https://raw.githubusercontent.com/coin-or/jorlib/refs/heads/master/jorlib-core/s
17
18
19 #Objeto de tsplib95 para nuestro problema problema
20 problem = tsplib95.load(file)
21
```

$(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9), (0, 10), (0, 11), (0, 12), (0, 13)$

```

15
16     return poblacion_final

```

```

1 #Funcion para generar hijos a partir de 2 padres:
2 # Se elige el metodo de 1-punto de corte pero es posible usar otros n-puntos, uniforme, dependiendo del prob
3 def Descendencia(padres, problem,mutacion):
4     pc = random.sample(range(1, len(padres[0])), 1)[0]
5
6     hijo1 = Factibilizar(padres[0][:pc] + padres[1][pc:], problem)
7     hijo2 = Factibilizar(padres[1][:pc] + padres[0][pc:], problem)
8
9     return [hijo1, hijo2, Mutar(hijo1, mutacion), Mutar(hijo2, mutacion)]
10

```

```

1 #Para el operador de cruce 1-punto los hijos generados no son soluciones(algunos nodos se repiten y otros no
2 def Factibilizar(solucion,problem):
3     Nodos = list(problem.get_nodes())
4     Nodos_desaparecidos = list(set(Nodos) - set(solucion))
5
6     for i in range(len(solucion)):
7         if solucion[i] in solucion[i + 1:]:
8             # Esta repetido
9             solucion[i] = Nodos_desaparecidos.pop(0)
10
11     return solucion

```

```

1 #Funcion de mutación. Se eligen dos nodos y se intercambia. Se podrian añadir otros operadores
2 # Se hace mutaciones mutacion% de las veces
3 def Mutar(solucion,mutacion):
4     if random.random() < mutacion:
5         sel1, sel2 = sorted(random.sample(range(1, len(solucion)), 2))
6         solucion_mutada = solucion[:]
7         solucion_mutada[sel1], solucion_mutada[sel2] = solucion_mutada[sel2], solucion_mutada[sel1]
8         return solucion_mutada
9     else:
10         return solucion[:]

```

```

1 #Funcion de seleccion de la población. Recibe como parametro una poblacion y
2 # devuelve una poblacion a la que se ha eliminado individuos poco aptos(fitness alto) y para mantener una po
3 #Se tiene en cuenta el porcentaje elitismo pasado como parametro
4 # Para los individuos que no son de la elite podríamos usar una selección de ruleta(proporcional a su fitnes
5 def Seleccionar(problem,poblacion, N, elitismo):
6     # Se ordena la población según el fitness(tamaño del recorrido) en una lista de elementos [distancia, so
7     poblacion_ordenada = sorted([(distancia_total(solucion, problem), solucion) for solucion in poblacion],
8
9     # Devolvemos elitismo% y el resto se eligen aleatoriamente
10    return [x[1] for x in poblacion_ordenada][:int(N*elitismo)] + random.sample([x[1] for x in poblacion_ord

```

✓ Proceso Principal

```

1 #Funcion principal del algoritmo genetico
2 #####3
3 def algoritmo_genetico(problem=problem,N=100,mutacion=.15,elitismo=.1,generaciones=100):
4     # problem = datos del problema
5     # N = Tamaño de la población
6     # mutacion = probabilidad de una mutación
7     # elitismo = porcion de la mejor poblacion a mantener
8     # generaciones = nº de generaciones a generar para finalizar
9
10    #Genera la poblacion inicial
11    Nodos = list(problem.get_nodes())
12    poblacion = generar_poblacion(Nodos,N)
13
14    #Inicializamos valores para la mejor solucion
15    (mejor_solucion, mejor_distancia) = Evaluar_Poblacion(poblacion, problem)
16
17
18    #Condicion de parada
19    parar = False
20    n=0
21    #Iniciamos el ciclo de generaciones
22    # NOTA: Se puede cambiar por un bucle for, y usar generaciones
23    # for n in generaciones
24    while(parar == False) :
25
26        #Cruce de la poblacion(incluye mutación)
27        poblacion = Cruzar(poblacion, mutacion, problem)

```

```

28
29     #Seleccionamos la población
30     poblacion = Seleccionar(problem,poblacion, N, elitismo)
31
32     #Evaluamos la nueva población
33     (mejor_solucion, mejor_distancia) = Evaluar_Poblacion(poblacion, problem)
34
35     print("Generacion #", n, "\nLa mejor solución es:" , mejor_solucion, "\ncon distancia " , mejor_distancia)
36
37     #Numero de generaciones. Criterio de parada
38     if n==generaciones:
39         parar = True
40         n +=1
41
42     return mejor_solucion
43
44
45 sol = algoritmo_genetico(problem=problem,N=500,mutacion=.3,elitismo=.40,generaciones=250)

```

Generacion # 0
La mejor solución es: [0, 3, 35, 36, 37, 17, 21, 39, 24, 29, 38, 9, 10, 5, 16, 32, 27, 26, 12, 28, 25, 40, 8, 2]
con distancia 3686

Generacion # 1
La mejor solución es: [0, 3, 35, 36, 37, 17, 21, 39, 24, 29, 38, 9, 10, 5, 16, 32, 27, 26, 12, 28, 25, 40, 8, 2]
con distancia 3686

Generacion # 2
La mejor solución es: [0, 1, 4, 6, 13, 5, 33, 14, 17, 3, 24, 21, 7, 34, 12, 23, 25, 11, 27, 15, 31, 36, 16, 29]
con distancia 3664

Generacion # 3
La mejor solución es: [0, 2, 3, 33, 5, 37, 7, 10, 8, 39, 14, 16, 17, 18, 13, 12, 23, 29, 9, 27, 34, 38, 31, 28]
con distancia 3654

Generacion # 4
La mejor solución es: [0, 1, 6, 35, 36, 3, 7, 5, 2, 14, 37, 15, 4, 22, 9, 11, 13, 29, 30, 12, 19, 26, 20, 33, 3]
con distancia 3547

Generacion # 5
La mejor solución es: [0, 3, 2, 10, 5, 38, 39, 9, 12, 18, 13, 16, 7, 8, 25, 14, 17, 31, 37, 27, 23, 33, 34, 28]
con distancia 3364

Generacion # 6
La mejor solución es: [0, 2, 32, 33, 34, 30, 28, 3, 13, 4, 40, 6, 5, 38, 22, 24, 39, 29, 41, 12, 21, 23, 25, 10]
con distancia 3198

Generacion # 7
La mejor solución es: [0, 2, 32, 33, 34, 30, 28, 3, 13, 4, 40, 6, 5, 38, 22, 24, 39, 29, 41, 12, 21, 23, 25, 10]
con distancia 3198

Generacion # 8
La mejor solución es: [0, 2, 32, 33, 34, 30, 28, 3, 13, 4, 40, 6, 5, 38, 22, 24, 39, 29, 41, 12, 21, 23, 25, 10]
con distancia 3198

Generacion # 9
La mejor solución es: [0, 2, 32, 33, 34, 30, 28, 3, 13, 4, 40, 6, 5, 38, 22, 24, 39, 29, 41, 12, 21, 23, 25, 10]
con distancia 3198

Generacion # 10
La mejor solución es: [0, 2, 32, 33, 34, 30, 28, 3, 13, 4, 40, 6, 5, 38, 22, 24, 39, 29, 41, 12, 21, 23, 25, 10]
con distancia 3198

Generacion # 11
La mejor solución es: [0, 32, 4, 33, 34, 36, 7, 3, 6, 39, 40, 9, 41, 12, 8, 10, 11, 14, 17, 27, 30, 25, 29, 1, 1]
con distancia 3193

Generacion # 12
La mejor solución es: [0, 33, 34, 32, 37, 5, 4, 8, 38, 10, 41, 11, 21, 39, 23, 24, 40, 29, 6, 7, 16, 19, 26, 13]
con distancia 3066

Generacion # 13
La mejor solución es: [0, 33, 34, 32, 37, 5, 4, 8, 38, 10, 41, 11, 21, 39, 23, 24, 40, 29, 6, 7, 16, 19, 26, 13]
con distancia 3066

Generacion # 14
La mejor solución es: [0, 33, 34, 32, 37, 5, 4, 8, 38, 10, 41, 11, 21, 39, 23, 24, 40, 29, 6, 7, 16, 19, 26, 13]
con distancia 3066

